Restaurant Service Robot Project Objectives

1. Navigation and Motion Objectives

Objective 1: Map-based autonomous navigation inside the restaurant without floor lines

The robot should be able to travel from the kitchen to any table in the restaurant and back, using a digital map of the space that it builds once by itself, with no need to paint lines or stick markers on the floor.

The positions of the tables, the kitchen point and the charging point are stored on the map as waypoints, so it is enough to give the robot a table number for it to plan its path and get there. If the restaurant layout changes, the map is rebuilt or the table positions are edited in software only.

The literature review showed that RoboServe's line-following navigation is inflexible, since any change to the table layout means redrawing the lines, while the first paper proved that free navigation is possible on low-cost hardware. This objective moves the project from the level of the second paper to the level of the first.

This objective includes:

- Using ROS 2 as the robot's software framework.
- Using a low-cost 2D LiDAR (such as an RPLiDAR A1 or equivalent) to build the map.
- Building the map with an off-the-shelf SLAM algorithm (such as slam_toolbox), then saving it and using it for localisation during operation.
- Planning the global path with A* (or its equivalent in the Nav2 stack), as in the first paper.
- Storing each table's coordinates in an easily edited configuration file.

Objective 2: Safe avoidance of static and moving obstacles

The robot should detect any obstacle that appears in its path, whether static, such as a chair left in the aisle, or moving, such as a waiter or customer walking by, and stop, slow down or change its path without colliding.

If the obstacle is a person standing in the way, the robot gives a polite voice alert, waits briefly, and then looks for an alternative path if the person does not move.

RoboServe's detour uses fixed timed moves with no map or distance measurement, which is unsafe in a crowded restaurant. The first paper used DWA for local obstacle avoidance but did not measure obstacle-detection accuracy. This objective therefore combines real local planning with the voice-alert idea from the second paper, while actually measuring performance.

This objective includes:

- Using a Nav2 local planner (such as DWB, a development of DWA) driven by live LiDAR readings.
- Adding a continuously updated obstacle layer (costmap) with a safety margin around every obstacle.
- Implementing the logic: detect a person in front of the robot, then give a voice alert, then wait 5 seconds, then replan.
- Optional addition: a camera with a recent YOLO model to distinguish people from objects and adapt the behaviour to the type of obstacle.

Objective 3: Accurate robot localisation through sensor fusion

The robot should know its position and heading inside the restaurant accurately at all times, even when the wheels slip or people block part of the LiDAR's view.

This is achieved by fusing several sources: wheel encoders that measure distance travelled, an IMU that measures rotation, and matching LiDAR readings against the map.

Any localisation error leads directly to stopping at the wrong table or getting too close to obstacles. The first paper showed that fusing an IMU with an EKF clearly reduces drift, an idea that can be applied with ready-made packages.

This objective includes:

- Fitting motors with encoders and an IMU (such as an MPU6050 or BNO055).
- Fusing wheel and IMU data with an EKF using the robot_localization package.
- Correcting the position on the map using AMCL or the localisation mode of slam_toolbox.

Objective 4: Precise and smooth motor control

The wheels should track the commanded speed quickly and accurately, and the robot should start and stop gradually and smoothly.

Sudden acceleration or vibration means drinks and soup spilling from the tray, so speed control here is not only a technical objective but a condition for service quality.

The first paper reduced settling time from 0.58 to 0.26 seconds by tuning the controller with a Genetic Algorithm. But it also showed that reducing this time can slightly increase overshoot, which matters for a robot carrying liquids.

This objective includes:

- Deriving a mathematical model of the motor (transfer function) from a simple response test, as the first paper did.
- Designing a PID controller on a microcontroller (such as an ESP32) for each wheel, tuning it manually first and then with a Genetic Algorithm using the ITAE criterion if time allows.
- Setting maximum acceleration and speed limits in the navigation configuration.

2. Service System Objectives

Objective 5: An integrated digital ordering system linking the customer, the kitchen and the robot

The customer orders from their table by scanning a QR code specific to it, which opens a web page with the menu. The order reaches the kitchen screen instantly with the table number. When the food is ready, the chef presses a button and a command is sent to the robot to go to that table.

In this way the robot becomes part of the restaurant's real workflow rather than just a cart operated by a person.

The second paper is the only one of the three to offer a complete service cycle, while the first paper has no ordering system at all. This objective takes the best of RoboServe and implements it with modern technologies.

This objective includes:

- A different QR code for each table, carrying the table number in the link.
- A simple web application (such as Flask or Node.js) with a database (such as SQLite or MySQL).
- A kitchen dashboard showing orders by arrival time and status (new, in preparation, ready, delivered).
- Connecting the server to the robot over the local network (MQTT or a ROS 2 bridge such as rosbridge).
- Issuing an electronic bill for each table, with the option of adding online payment later depending on the methods available locally.

Objective 6: Delivering more than one order per trip

The robot should carry several tables' orders in one trip on more than one shelf, visit the tables in the shortest possible order, and then return to the kitchen.

The robot knows which shelf belongs to which table and shows this on its screen at every stop, so the customer takes the right order.

One of RoboServe's clearest weaknesses is that it delivers only one order and then returns, which multiplies the number of trips at peak times. Delivering several orders is what commercial restaurant robots actually do.

This objective includes:

- A body with at least three shelves.
- A simple algorithm to order the tables by distance on the map (nearest table first, or an exact solution since the number of tables is small).
- Displaying the shelf number and table number on the screen at every stop.

Objective 7: Confirming food pickup and returning automatically

The robot should not leave the table until it is sure the customer has taken their food, and should then return automatically to the kitchen or move on to the next table.

Confirmation uses two methods together: a sensor that detects the shelf is empty, and a "Received" button on the screen. If the order is not collected within a set time, the robot sends an alert to the staff.

Empty-tray detection in RoboServe is a very simple and effective idea (97% success), whereas the first paper has no way of knowing when its task is finished. Combining a sensor with a button makes confirmation more reliable.

This objective includes:

- A load cell (with an HX711) under each shelf, which is more accurate than an IR sensor because it knows the food has actually been taken.
- A confirmation button on the screen as an alternative.
- An adjustable waiting timeout (for example two minutes), followed by an alert on the kitchen dashboard.

3. Intelligence and Interaction Objectives

Objective 8: Intelligent task planning with a large language model on low-cost hardware

The robot should handle unexpected situations flexibly, such as a customer asking for something extra, cancelling part of an order, or asking for a dish to be taken back. Instead of programming every case by hand, the robot's work is broken into simple skills, and a large language model (LLM) chooses the next skill according to the situation.

Examples of skills: go to a table, wait for pickup, return to the kitchen, send a message to staff, show a message to the customer. The LLM does not control the motors directly; it only chooses from this list.

The third paper showed that this approach makes the robot flexible in the face of changing orders, but it applied it to a very expensive HSR robot. Applying it to a low-cost robot fills a clear gap revealed by the comparison of the three papers.

This objective includes:

- Defining a limited, clear list of skills, each a ROS 2 service or action.
- Calling an LLM through an API from the central computer or server, not on the robot itself, to save computing power.
- Constraining the model's output to a fixed format (such as JSON) and validating it before execution, rejecting any skill not on the list.
- A fallback mode: if the internet drops or the model is slow, the robot runs a fixed pre-programmed sequence so that service does not stop.

Objective 9: One-button correction of robot mistakes during operation

The robot's screen should have a "Something went wrong" button that a customer or waiter presses if they notice the robot has stopped at the wrong table or brought the wrong order. The system then analyses the situation and decides on the appropriate correction, such as moving to the correct table or returning to the kitchen to replace the order.

The person pressing the button does not need to explain the problem, because the system knows what the robot has done so far and what it was asked to do.

In the third paper this button raised the success rate from 55% to 86% and reduced customer anxiety, even though it was used in only 8 of 50 tasks. These are errors the robot cannot detect on its own, such as the chef placing the wrong dish on the shelf.

This objective includes:

- A clear button on the screen, and a matching button on the kitchen dashboard for staff.
- On press: sending the log of executed skills and the original order to the LLM so that it proposes a correction plan.
- If the model is unavailable: automatically calling a member of staff.

Objective 10: A clear interactive interface showing the robot's state

Customers and staff should know at any moment what the robot is doing and what it will do next, through a screen on the robot that shows its state in a short, clear message (such as "Heading to table 4" or "Waiting for you to take your order").

The screen is accompanied by short voice messages on arrival, when there is an obstacle and on departure, in the restaurant's language. Lighting with colours indicating the state can also be added.

The third paper observed that some customers, especially children, did not intervene when an error occurred because they did not understand what the robot was doing, and recommended showing its state clearly. The second paper showed the value of voice and touch interaction.

This objective includes:

- A touchscreen (7 inches or larger) or a tablet mounted on the robot displaying a web page.
- Pre-recorded voice messages or messages generated with text-to-speech (TTS).
- An LED strip with colours indicating the state (moving, waiting, error, charging).

4. Hardware and Safety Objectives

Objective 11: Low-cost hardware design using available components

The robot should be built from components available on the local market or easy to buy, at a total cost far below commercial restaurant robots, with a detailed table of every component and its price.

Processing should be divided sensibly: heavy computation (such as the LLM) on an external server or computer, navigation on a small computer on the robot, and motor control on a microcontroller.

The comparison of the three papers showed that intelligence rises with cost, and that the real challenge is delivering the third paper's intelligence on hardware close to that of the first and second. The first paper also did not explain how the Raspberry Pi coped with the computational load, so this must be measured here.

This objective includes:

- A small onboard computer (Raspberry Pi 4/5 or equivalent) to run ROS 2 and navigation.
- A microcontroller (ESP32 or Arduino) for the motors and sensors.
- A low-cost LiDAR, motors with encoders, an IMU and load cells.
- A bill of materials (BOM) with prices, compared against the price of commercial robots.

Objective 12: A sturdy body, payload and battery suited to real work

The robot should have a rigid, stable body (for example aluminium or reinforced acrylic) that carries several dishes and drinks at once, with a low centre of gravity so it does not shake or tip over.

The battery should last a full service period (such as lunch or dinner) without recharging.

RoboServe's body is made of wood and paper with a payload of only 1.5 kg, and its durability was never tested, which does not suit a real restaurant. And because a heavier payload increases power consumption, battery runtime must be measured with a full payload, not without one.

This objective includes:

- Designing the body in CAD software and checking its strength before manufacturing.
- Placing the battery and motors at the bottom to lower the centre of gravity.
- Choosing a lithium battery of suitable capacity, with a battery management system (BMS) and a charge indicator on the screen.

Objective 13: Safety while working among people

The robot should be completely safe for customers, staff and children: it must not exceed a safe speed, it must stop immediately when needed, and it must have no sharp edges.

The robot works in a crowded space among people who do not anticipate its movement. Safety is the first thing an assessment committee or any restaurant owner will ask about, and none of the three papers tested it systematically.

This objective includes:

- A clearly visible emergency stop (E-Stop) button that cuts power to the motors.
- Limiting maximum speed to about 0.5 m/s and reducing it near tables.
- A front bumper with a contact sensor as a last line of protection if the LiDAR fails.
- Rounded edges on the body.